

SmartSailor — Judge Q&A Sheet

One platform, two workflows, one intelligent layer. | Every answer below is grounded in the shipped codebase. Each is written to be spoken in 20–40 seconds. **M1** = Salon **M2** = Beauty Parlour **M3** = AI & Architecture

THE THREE SENTENCES EVERY MEMBER MUST BE ABLE TO SAY: (1) Salon = live queue, parlour = appointments — one codebase, two rules, and the rule lives in `src/lib/business-model.ts`. (2) The AI proposes, a human confirms, the database decides — 25 tools, not one of them can write. (3) Double booking is blocked by a PostgreSQL exclusion constraint, not by application code.

A · PRODUCT & PROBLEM

Why this problem?

In a small salon you cannot find out how long the wait is without physically going there. The problem is not the waiting — it is the not knowing. And on the owner's side the queue lives in a paper register and the maths lives in his head, so a chair sits empty while a customer waits outside. The relevant comparison is not an international booking app; it is a notebook.

■ M1

Why two different workflows?

Because a haircut is fifteen minutes and nobody books a time for it — they walk in and take a number. A bridal package is three hours and is booked ten days ahead. “Position 4, about 25 minutes” is meaningless for the second; “choose a slot on Thursday” is meaningless for the first.

■ M1

Why not one universal booking system?

A universal system would be wrong for both — each business would get a form with fields it must ignore. So we split where the workflow genuinely differs (intake) and shared everything where the business is the same: payments, membership, loyalty, referral, rewards, reviews and analytics are identical for both verticals.

■ M1

How is this different from existing booking apps?

Two ways. Most of them only do appointments, so a barbershop has to pretend its walk-ins are bookings. And they are built for shops that already run software — ours is built for a three-chair shop whose current system is a notebook, in Bangla, on a phone.

■ M1

Who is the harder user — the customer or the owner?

The owner, and that is who we built first. A booking app with no shops is nothing. The owner needs queue, staff, money, loyalty and analytics in one place; the customer needs one number and one time.

■ M1

B · SALON / BARBERSHOP

How is queue position and wait time calculated?

Server-side in PostgreSQL on insert, from the real service durations and the chair assignment — never in the client. It recomputes whenever the queue changes, including when the owner starts a break, and the new value is pushed to the customer over Supabase Realtime.

■ M1

What if the shop closes while people are still in the queue?

There are two separate controls, deliberately. “Closed” stops everything; “not accepting new” stops intake while the existing queue is still served. Without that split an owner had to choose between taking serials he could not serve and vanishing from Explore with customers still sitting inside.

■ M1

What is the style picker, and is it on every shop?

It is per-shop, optional, and salon-only. The owner configures style options per service and the customer picks one at booking, so the barber gets the intent before the customer sits down. A parlour's appointment sheet has nothing to feed it, so it is not shown there.

■ M1

What about walk-in customers with no phone?

The owner adds them from the board and they take a position in the same queue. There is also a QR poster for the shop so a walk-in can add themselves. That is the honest limit of the product: someone at the shop needs a device.

■ M1

C · BEAUTY PARLOUR

How are the available time slots computed?

Four inputs, every time: the service duration, which staff can actually perform that service, that staff member's working hours for that weekday, and the appointments they already have — compared as a time-range overlap. Nothing is a stored slot list, which is why the gaps in the grid are real bookings rather than decoration.

■ M2

How do you prevent double booking? What if two customers book the same slot in the same second?

We do not prevent it in application code, because check-then-insert always leaves a race window. There is a PostgreSQL exclusion constraint — EXCLUDE USING gist on (staff_id, the appointment's time range) — so two overlapping appointments for one staff member cannot be inserted at all. One transaction wins; the other gets a constraint violation. We verified it with two genuinely parallel clients against real PostgreSQL.

■ M2 / M3

Why does the constraint have a status filter on it?

Because a cancelled or no-show appointment must not keep blocking the slot it used to hold. The constraint applies only to BOOKED, CONFIRMED and IN_PROGRESS.

■ M2

Can a parlour take an appointment while it is marked closed?

Yes, deliberately. The open switch is the queue's "taking walk-ins right now" flag. Refusing tomorrow's 10am booking because it is 11pm tonight would be absurd for an appointment business, so appointment_before_insert checks that the shop is active and ignores is_open — and the code comment says exactly that.

■ M2

Can a customer cancel or reschedule?

Not yet, and I will not pretend otherwise. The provider can move an appointment to CANCELLED or NO_SHOW, but customer-initiated cancellation and rescheduling are not implemented. They are the third item on our near-term list.

■ M2

D · AI — WHAT IT IS AND IS NOT

Why do you need AI here at all?

You do not, and that is on purpose — remove it and you still have a working product. It earns its place in two narrow spots: an owner who cannot read an analytics dashboard can ask a question in Bangla, and an owner who cannot easily see "twelve regulars have not come since July" now can.

■ M3

What exactly does the AI do?

Three things. It reads business data through 25 read-only tools and explains it in Bangla. It helps customers find shops, services and available times. And it drafts four kinds of action — join queue, book appointment, redeem reward, marketing campaign — as proposals a human must confirm. It never executes anything itself.

■ M3

Is this just an API call to Claude? Is it a chatbot?

No. A single call would mean either sending our database in the prompt or accepting invented answers. We run a bounded agent loop: the model picks tools from a closed registry, we execute them server-side through RLS-protected RPCs, fence the results as data and hand them back. Maximum four iterations, eight tool calls.

■ M3

How does the AI access our business data? Are you sending the whole database to the model?

It has no access to our data — it has access to questions it can ask. It calls get_overview or get_peak_slots; the server runs that analytics RPC as the authenticated owner, and the shop id comes from the owner's own row, never from the request and never from the model. For marketing it gets a count of customers, never the list.

■ M3

Can the AI directly modify the database?

No. There is no write tool in the registry — that is checked at import time and a test asserts it. The only mutation path is a CONFIRMED row in our ai_actions table plus a human click, and the endpoint that performs it does not call Claude at all.

■ M3

How do you prevent hallucinated shop or service IDs?

We keep a ledger of the IDs a tool actually returned in this conversation. An ID that was not in a real tool result is refused — even a well-formed UUID that happens to exist. Then the server independently re-verifies the row, and that the services belong to that shop.

■ M3

How do you prevent cross-business data leakage?

Four layers. The role is derived from a database query, not claimed in the request. The shop id comes from the owner's own row. Every segment function re-checks is_shop_owner() inside SQL and raises "not your shop". And the client is cookie-bound, never service-role, so RLS applies to everything. Cross-shop attacks are explicit cases in our PostgreSQL harnesses.

■ M3

How do you prevent prompt injection?

Every tool result is fenced before the model sees it, labelled as data with an instruction never to follow instructions inside it — which matters because shop names and review text are written by the public. And structurally, even a successful injection cannot do damage: there is no tool to hijack. The worst case is a wrong sentence, not a wrong write.

■ M3

What happens if the price changes after the AI proposes an action?

The proposal stores the price it showed. At confirm time the server re-reads the current price and refuses with PRICE_CHANGED rather than charging a different amount.

■ M3

What if another customer takes the slot, or the customer segment changes, before confirmation?

The confirm goes through the same RPC and the same exclusion constraint as a button press, so whoever commits first wins. For a campaign the proposal stores a snapshot of exactly who will receive it, not a segment name — if the audience moved we refuse with SEGMENT_CHANGED rather than silently messaging a different group.

■ M3

What if the user double-clicks Confirm?

An atomic state transition claims the action. We tested five genuinely simultaneous clients against real PostgreSQL: exactly one execution. For campaigns a unique index adds a second guard, so even a retried request after a lost response cannot send twice.

■ M3

Which model, and why that one?

claude-opus-5, server-side, through the official Anthropic SDK. The job is reasoning over several tool results and declining to invent numbers — a wrong figure costs a shopkeeper real money. Each feature has its own environment-variable override so any one of them can be moved to a cheaper model without a release.

■ M3

Why server-side AI and not from the browser?

Three reasons: the API key would be in the bundle and anyone could spend it, the model would inherit whatever database access the browser has, and we could not enforce the tool registry — the whole safety property depends on the server deciding what is callable.

■ M3

How do you control AI cost?

Structurally: four loop iterations, eight tool calls, twelve turns of history at two thousand characters each, and an output ceiling per feature. Honestly, though — there is no per-user rate limit or daily spend cap yet. That is the first thing we would add before real traffic, and we know where it goes.

■ M3

What happens if the Anthropic API is unavailable?

The AI widget shows an error and nothing else notices. No booking, queue, payment or analytics feature calls Claude. The AI is a layer on top of a working product, not the product.

■ M3

Which parts are AI and which are deterministic?

Everything that matters is deterministic. Queue position, ETA, slot availability, conflict prevention, pricing, loyalty points, referral qualification, analytics figures and customer segments are all SQL. The AI chooses which question to ask and writes the Bangla sentence around the answer. Segment membership in particular is a WHERE clause, never a model judgement.

■ M3

Why should we trust your AI?

You should not, and the system does not either — that is the design. Every number traces to a row, every segment is a SQL rule, and every change needs a human to press a button. The trust sits in PostgreSQL and in the owner, not in the model.

■ M3

Why didn't you train your own model, or use an open-source one?

Nothing here needs a trained model. The predictions people expect from ML — demand, no-shows, churn — we deliberately did not build, because we do not have the data to make them honest. As for open-source: the model sits behind one client module and a closed tool registry, so swapping provider is a configuration change. Nothing about the safety design depends on who serves it.

■ M3

E · TECHNOLOGY CHOICES

Why PostgreSQL, and not MongoDB?

Because our hardest problem is relational and concurrent: does this beautician already have an appointment overlapping this time range? In PostgreSQL that is one exclusion constraint and the database refuses the row. In MongoDB there is no equivalent — we would be doing read-then-write in application code, which always leaves a race window, and we would have rebuilt weaker versions of transactions, foreign keys and constraints ourselves.

■ M3

Why Supabase, and not raw PostgreSQL or Firebase?

Supabase is PostgreSQL — we write real SQL migrations and own the schema, so there is no lock-in on the data model. It adds three things we would otherwise build: auth whose session RLS can read as `auth.uid()`, realtime off logical replication, and storage. Firebase would have meant giving up SQL constraints and transactions, which is exactly where our correctness lives.

■ M3

Why Row Level Security instead of checking permissions in the API?

Authorization belongs next to the data. With RLS a forgotten `shop_id` filter returns zero rows instead of another shop's customers, and the check cannot be bypassed by a new endpoint, a new query or the AI layer — they all go through the same policies. It is also testable on its own: our harnesses connect as different roles and assert what each can see.

■ M3

Why Next.js?

One deployable and one security boundary. The Anthropic key and the service-role key live in Route Handlers, and a "server-only" import makes reaching them from the browser a build error rather than a runtime leak. A separate SPA plus API server would have meant two deployments and two places to leak a key.

■ M3

Why TypeScript?

We generate types from the database schema. With 149 tables and views, a renamed column becomes a compile error instead of something a customer discovers.

■ M3

How do you protect the API key? How do you prevent SQL injection?

The key is read only inside modules marked "server-only", so a client import fails the build — same for the service-role key. For SQL: there is no string-built SQL anywhere. The client is a query builder and the server logic is parameterised PL/pgSQL. The AI has no SQL surface at all.

■ M3

F · SCALE, LIMITS AND ROADMAP — ANSWER THESE HONESTLY

Posture on every question in this section: agree with the premise, give the specific fact, name the limit without defending it. Judges have heard a hundred inflated claims today. “We have not measured that” makes every other claim you make more credible.

How does this scale to thousands of users?

The read patterns are indexed and shop-scoped, so they partition naturally. The honest answer is that we have not load-tested. What we have tested is correctness under concurrency, with parallel clients against real PostgreSQL — which is the failure we thought would bite first. Our likely first bottlenecks are Realtime connections per shop on the queue board, and the AI endpoint.

■ M3

Have you tested this with real users? How many shops use it?

None yet. It is built and tested — 1,159 unit tests and 14 PostgreSQL concurrency harnesses — but not deployed to real shops. I would rather tell you that than quote a user number.

■ M1

What is your biggest technical limitation?

Two. We have no load testing, so we can speak to correctness but not throughput. And the AI endpoint has no rate limit or spend cap, so one user could run up cost. Both are known, both have a clear fix, neither is hidden.

■ M3

Isn't this just CRUD with extra steps?

A lot of it is CRUD, and I will not pretend otherwise. The parts that are not: queue position and ETA recomputing across chairs, slot availability derived from duration and staff capability and existing bookings, conflict prevention by exclusion constraint, and the AI proposal architecture. That is where the engineering went.

■ M3

What is the business model?

A per-shop subscription is the natural fit, with the AI features as a higher tier since they carry real per-call cost. Nothing is implemented — there is no billing in the product — so I will not present a revenue model as if it were built.

■ M1

What is the roadmap?

In order: apply our three pending AI migrations, add an AI rate limit and spend cap, then customer-initiated cancellation and rescheduling. Those are gaps we know about rather than ideas. Further out: demand forecasting, no-show prediction and churn scoring — but we deliberately did not build prediction, because we do not have the data to make it honest. There is no churn score anywhere in this product today.

■ M3

Can a customer use one account across both kinds of business?

Yes — one account, one identity. At registration they choose which service they came for and the app opens as that app. It is a client-side preference, not an access rule: a shop page reached by link or QR still loads. Today changing it means going to settings, and a cleaner answer is on the roadmap.

■ M1

G · DO NOT CLAIM — READ BEFORE GOING ON STAGE

Never say	Say instead
AI predicts demand / no-shows / churn	We deliberately do not predict — we describe what happened
Our AI learns from your business	No training, no memory, no embeddings — it reads live data through tools
AI sends campaigns automatically	AI drafts; the owner edits and approves; then the server sends
We have N users / N shops	Not deployed to real shops yet — built and tested
Production-ready, scales to millions	Verified by 1,159 tests and 14 concurrency harnesses; not load-tested
We use machine learning	A language model for understanding and wording. No ML model of our own.
X% accurate	Nothing — we have measured no accuracy figure
Customers can cancel or reschedule	Provider-side status changes today; customer-initiated is roadmap

SMS / WhatsApp / email notifications	In-app and web push. No external messaging provider.
Real-money payments	Cash and due tracking. No payment gateway.
AI cost is capped per user	Bounded loop and output ceilings — no per-user rate limit yet

When you do not know a number, say “I don't have that number” — then say what you do have.

H · VERIFIED FACTS — MEMORISE, DO NOT INFLATE

67	SQL migrations
149	tables and views
1,159	unit tests (48 files)
14	PostgreSQL concurrency harnesses, with genuinely parallel psql clients
25	AI tools in the registry — every one read-only
4	AI action types: join queue, book appointment, redeem reward, send campaign
4 / 8	MAX_ITERATIONS / MAX_TOOL_CALLS in the agent loop
12 × 2000	turns of history × characters per message, capped

Stack: Next.js 16.2.10 (App Router) · React 19.2.4 · TypeScript 5 · Tailwind CSS 4 · TanStack Query 5 · Zod 4 · Leaflet · PostgreSQL + Row Level Security via Supabase · Supabase Auth (PKCE) · Supabase Realtime · @anthropic-ai/sdk 0.121 with claude-opus-5 · Vitest 4 · Playwright · ESLint 9 with eslint-plugin-boundaries · Vercel + Vercel Cron